

QCon
全球软件开发大会

让开关自我消亡： AI 赋能的 Feature Flag 全生命周期治理

闫文亮

快手 资深服务端架构师

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

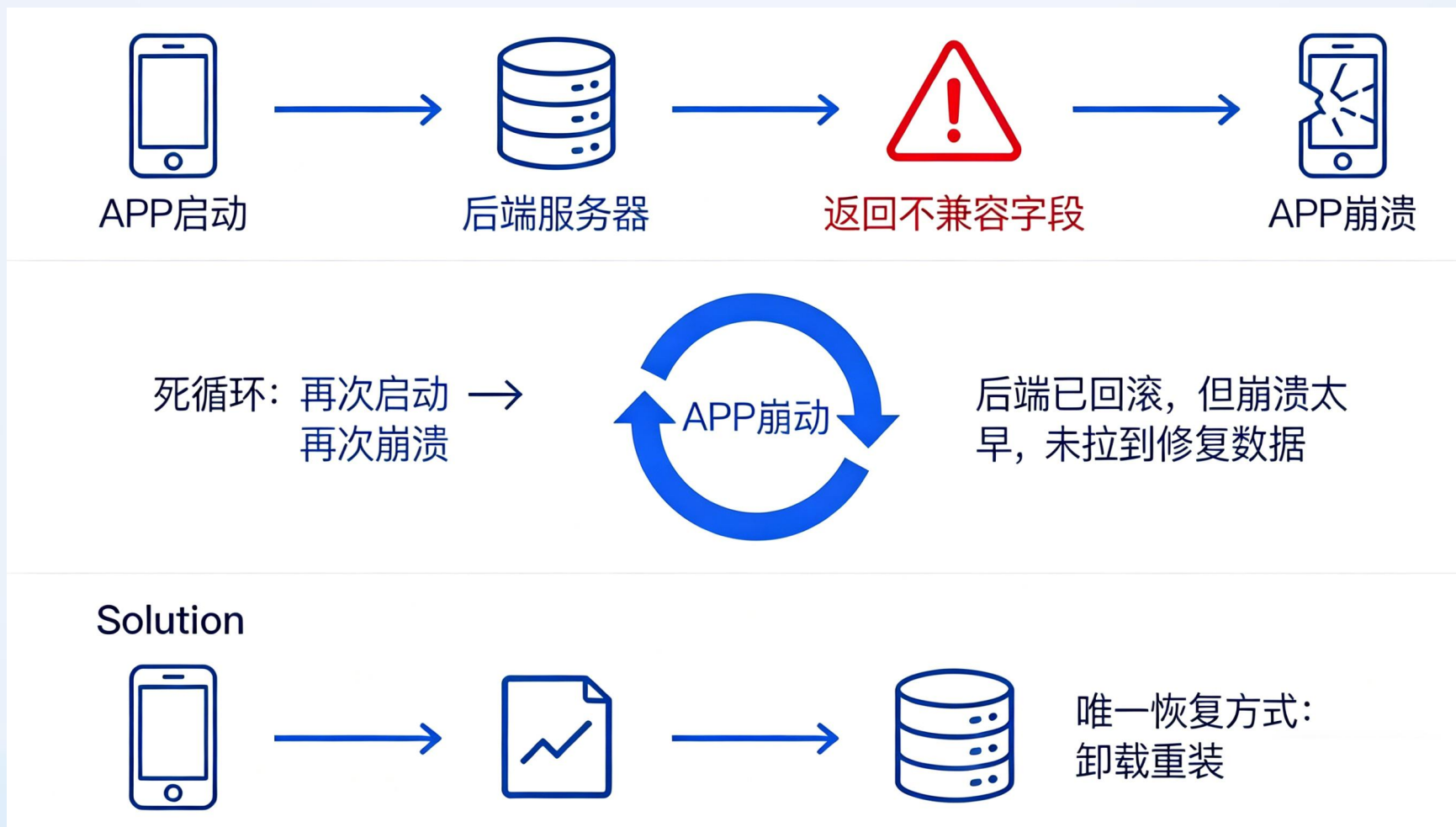
- 01 Feature Flag价值与隐形技术债
- 02 AI治理Agent: 从Demo到工程落地
- 03 安全护栏: 三重校验 + 双引擎架构
- 04 自进化: 双Agent驱动系统自愈
- 05 总结与展望

01

Feature Flag价值与隐形技术债

从业务刚需到技术债陷阱

真实的故障



如果你是这个需求的研发, 你的心情会怎么样?

什么是Feature Flag

无Feature Flag: 全量发布

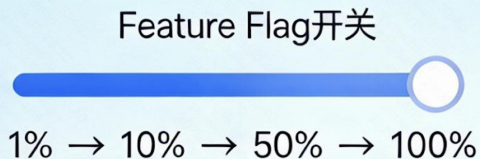


有Feature Flag: 内部灰度



Feature Flag的价值

(灰度放量 - 安全阀门)



少量用户



大量用户

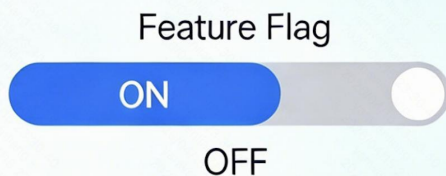
灰度放量：先让1%用户试用，确认无误后再全量

避免全量上线的风险

(快速回滚 - 降低损失)



功能异常



(秒级)



服务恢复

秒级关闭：发现问题立即关闭，无需等待代码回滚

快速回滚，降低损失

(发布解耦 - 核心对比)



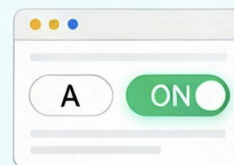
A

B

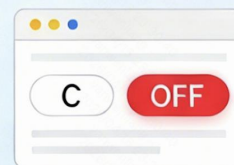
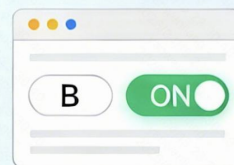
C

代码包A+B+C，B出问题全部回滚

Vs



A和C保持开启，只关闭B，各功能独立控制，互不影响



是我们迭代路上的好帮手

Feature Flag的价值

AI

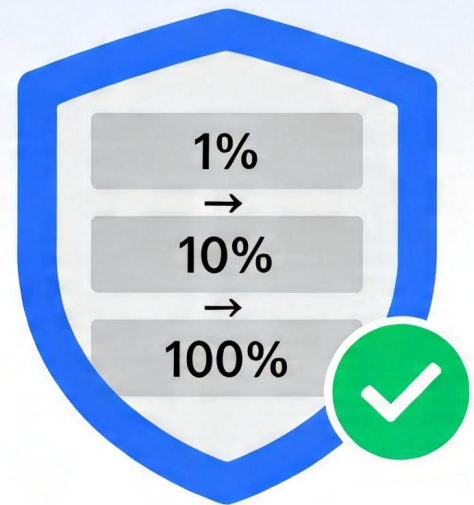


新功能发布风险增加

AI生成代码，研发人员不熟悉

更需要谨慎发布

Feature Flag/保护



1%
→
10%
→
100%

Feature Flag: 安全发布的守护者

AI Coding时代，不熟悉代码更要谨慎，
Feature Flag是必备安全网

AI生成



代码中几步之内必有开关

```
private boolean inject[...] onfig(...) {
    if (type != ...) {
        return false;
    }
    if (KswitchInstance.WEB_SERVER_POSTER.getBoolean( key: "enable...", defaultValue: false, ksFacts())) {
        TagScope.setTagTypeConfig(MAGIC_..._FIG_V2.get());
        perf( namespace: "search...", subtag: "v2").logstashOnly();
        return true;
    }
    if (KswitchInstance.WEB_SERVER_POSTER.getBoolean( key: "enable...", defaultValue: false, ksFacts())) {
        TagScope.setTagTypeConfig(MAGIC_..._IG_MAP.get().get(type.getTagType().getValue()));
        perf( namespace: "search...", subtag: "v1").logstashOnly();
        return true;
    }
    perf( namespace: "search...", subtag: "default").logstashOnly();
    return false;
}
```

```
public static boolean enable...teParams(String bizName) {
    KsFacts ksFacts = KsFacts.newBuilder(ksFacts())
        .custom("bizName", bizName)
        .build();
    return KswitchInstance.WEB_SERVER_SEARCH.getBoolean( key: "enabl...", defaultValue: false, ksFacts);
}
2 usages  luoxinpeng

public static boolean enable...imize() {
    return Kswitches.WEB_SERVER_SEARCH.getBoolean( key: "en...", defaultValue: false);
}

10 usages  hukui

public static boolean enable...plit() {
    return KswitchInstance.LOCAL_LIFE_PLATFORM.getScopeBoolean( key: "enabl...", defaultValue: false, ksFacts());
}
```

开关多带来的问题

代码维护成本飙升

新人接手项目，在一堆开关里梳理逻辑，分不清哪些有用、哪些没用

AI写代码后，我们都成了新人，对代码都不熟



浪费计算资源

每一次接口调用、每一次逻辑判断，都要经过这些无用开关的校验
日积月累就是不小的损耗

短视频服务，每秒会调用开关15亿次

浪费带宽资源

客户端使用的开关需要通过接口进行下发

每年开关下发的带宽成本就有xx万



隐藏稳定性风险

过期开关偶发拉不到推全的值，走到了旧的逻辑
引发意想不到的线上问题

过期开关没拉到推全的值



⚠️ 推全的开关也要及时清理, 否则就是定时炸弹

治理困境

加开关 - 容易

+ 加开关



时间

🚀❤️ 1分钟

```
if (flag.isOn()) { ... }
```

收益

- ✅ 解决上线风险
- ✅ 故障快速回滚
- ✅ 发版解耦



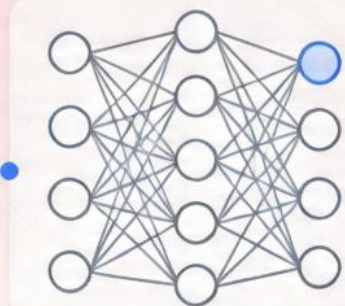
门槛极低，收益极高，顺手就做

清开关 - 困难

✓ 清开关

🚧❤️ 1小时+

1. 梳理上下游逻辑
2. 评估下线风险
3. 修改代码
4. 测试验证
5. 发布上线



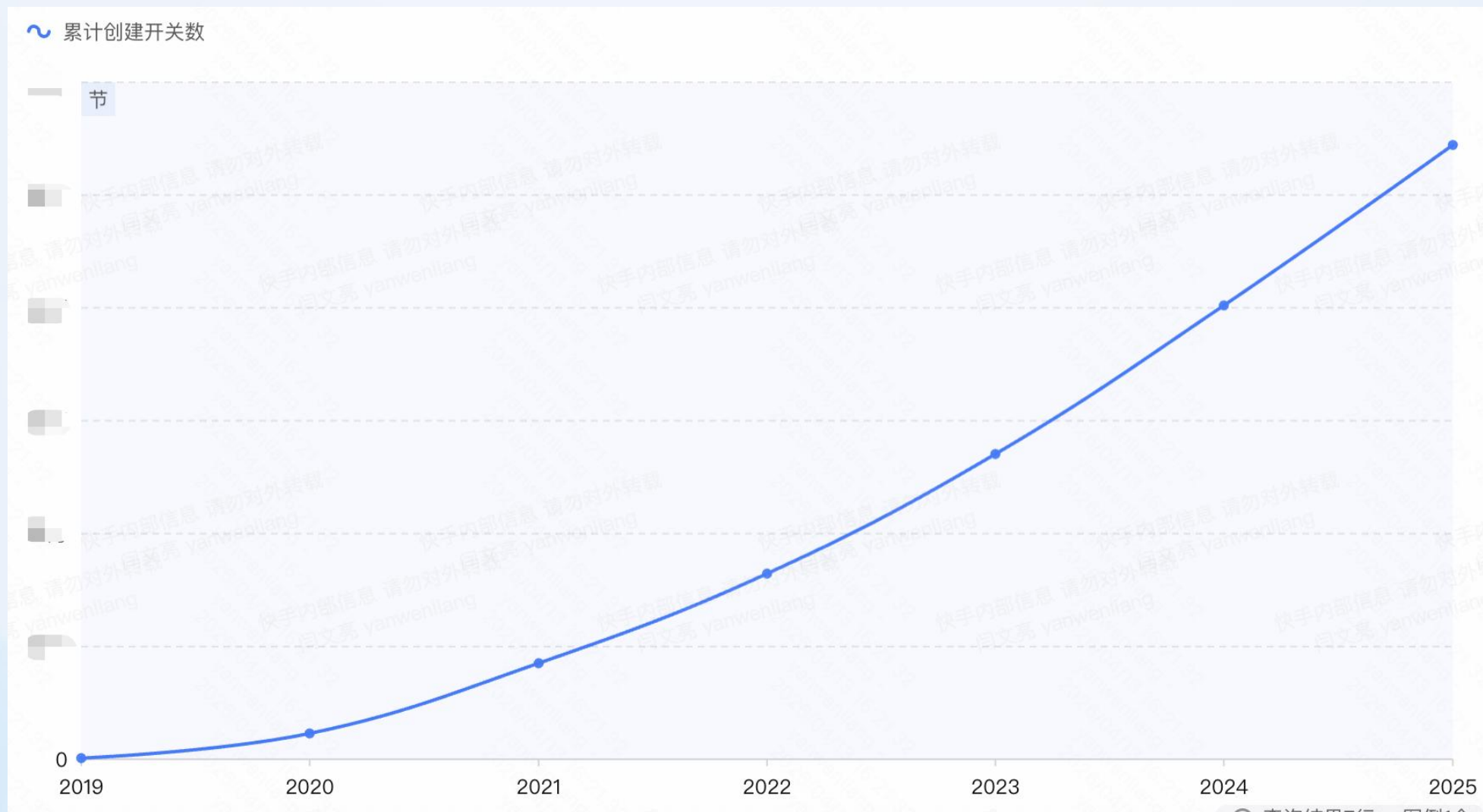
额外工作，无收益，担风险，没人愿意做

开关债务的根源：加法容易，减法困难

治理困境



治理困境





AI时代加速技术债累积

Technical Debt Accumulates at Unprecedented Scale

技术债以前所未有的规模累积

Armando Solar-Lezama, a professor at MIT specialising in program synthesis, offered a colourful assessment in remarks widely cited across the industry: AI represents a “brand new credit card here that is going to allow us to accumulate technical debt in ways we were never able to do before.”

麻省理工学院专注于程序合成的教授阿曼多·索拉尔-莱扎马在业内被广泛引用的言论中给出了一番生动的评价：人工智能就像一张“全新的信用卡”，将让我们以前所未有的方式累积技术债务。

“What we're seeing is that AI code assistants excel at adding code quickly, but they can cause 'AI-induced tech debt,’” explained GitClear founder Bill Harding. “This presents a significant challenge for DevOps teams that prioritise maintainability and long-term code health.”

“我们发现，AI 代码助手在快速编写代码方面表现出色，但它们可能会造成‘AI 引发的技术债务’，”GitClear 创始人比尔·哈丁解释道。“这对那些优先考虑可维护性和长期代码健康的 DevOps 团队来说，是一项重大挑战。”

传统治理方式的局限



平台规范治理

设置过期时间、搭建审计看板
依赖业务自觉性，执行参差不齐



工程脚本治理

自动化脚本扫描删除
灵活性不足，复杂场景易出错



专项治理行动

定期组织清理专项
耗时耗力，“一周风”式，无法持续



核心痛点

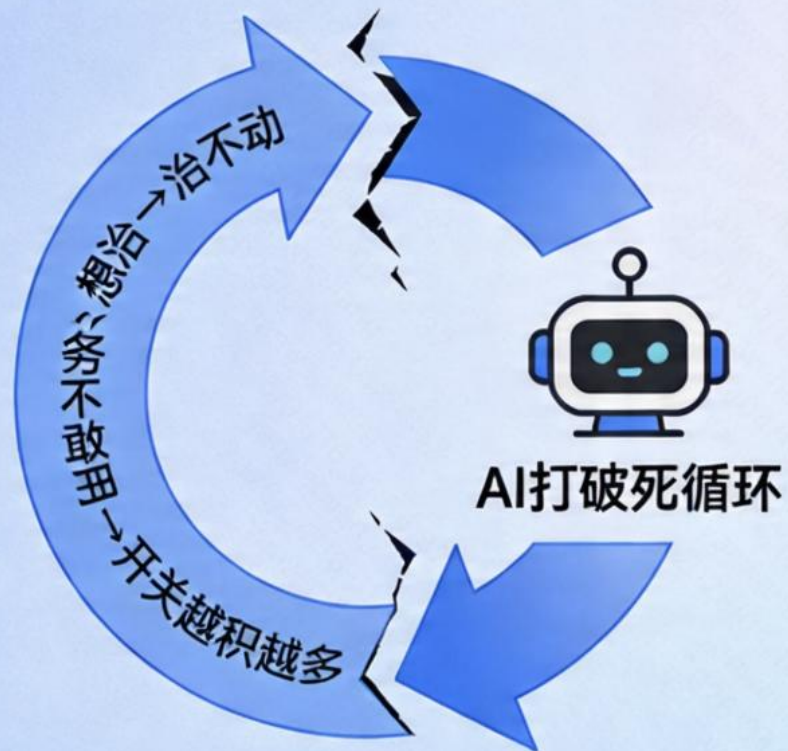
业务怕下线引发故障，
哪怕确认无用也不敢操作

想治→治不动
→业务不停用→
→开关越积越多→
→回到起点

治理速度远远跟不上开关新增速度

AI打破死循环

治理死循环



外部条件已成熟

1. 大模型技术快速迭代
2. API调用成本降低
3. 代码理解、修改能力成熟



内部条件已成熟

1. 开关治理痛点必须解决
2. 业务迫切需要高效安全治理方式
3. 公司AI基建不断投入

时机刚刚好



AI治理新时代

02

治理Agent的演进与工程落地

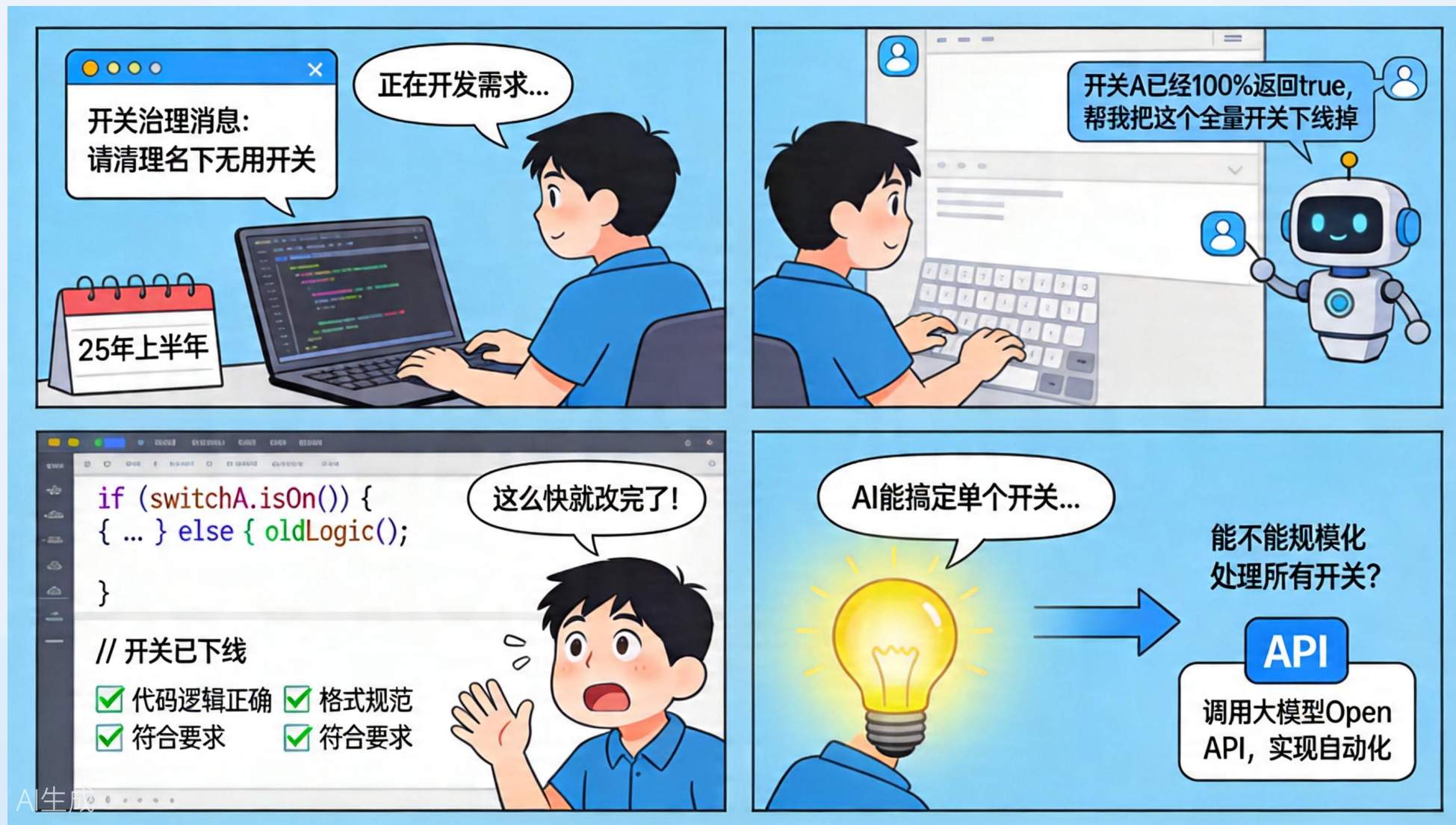
从Demo到工程落地

治理Agent的演进与工程落地

快手开关治理发展历程



AI治理开关初尝





治理Demo搭建



Bad Case

```
41      41      import com.google.common.util.concurrent.ListenableFuture;
42      42      import com.google.protobuf.ByteString;
43      - import com.google.common.util.concurrent.Futures;
44      - import com.google.common.util.concurrent.ListenableFuture;
45      43      import com.google.common.util.concurrent.ListenableFuture;
46      44      import com.google.common.util.concurrent.ListenableFuture;
```

```
er.filterAtInCommentVisible(filteredComments, userId, ...;
er.filterAtInCommentVisible(filteredComments, userId, ...;

erv. ...

ervice... userId);
```

提示词调优

5. **禁止修改非目标代码**:
 - 禁止修改其他开关调用: 即使使用相同的Facts对象, 也不能修改其他开关的调用逻辑
 - 禁止简化共享的方法调用: 如`ksFacts()`等被多处使用的方法调用, 即使在目标开关中被优化掉, 也不能简化
 - 禁止修改共享的变量构建: 如`KsFacts.newBuilder(ksFacts())`等包含上下文信息的调用, 不能简化
 - **禁止删除仍被其他代码使用的import**: 必须确认import在整个文件中完全不被使用才能删除
6. **逻辑等价性保证**:
 - 只有在确保不影响其他代码逻辑的前提下才能进行优化
 - **保持所有非目标开关的原有行为不变**
 - 保持共享组件 (Facts、Builder等) 的原有语义
7. **渐进式安全优化**:
 - **第一优先级**: 确保代码可编译, 不出现语法错误
 - **第二优先级**: 确保逻辑等价性, 不改变非目标代码行为
 - **第三优先级**: 在前两个条件满足的基础上进行代码优化

输出要求

```
<think>
{{思考的内容, think step by step}}
</think>
<code>
{{完整的优化后Java代码, 不包含任何markdown标识符、代码块标记或额外说明文字, 确保输出为纯净的Java源代码}}
</code>
```

思维链 (Cot)

逻辑错误引起足够重视



方法名与开关名一致

大模型误将方法当成开关
相关代码删除
整个功能模块失效



逻辑改反

“返回false时执行A” →
误改成“返回true时执行A”
业务逻辑异常



开关名混淆

开关A和B的Key相似，误
将B也下线
误下线正常开关



无关代码误改

修改开关时，把相邻的'if
(a == 1)'改成'if (a == 2)'
莫名其妙改坏业务逻辑



完全信任大模型
= 被动等待故障发生

不能指望大模型“永远不出错”



建立安全护栏
拦截所有错误

防止错误代码透出到线上

多轮对话



Andrej Karpathy 安德烈·卡帕西

@karpathy @kar 🤔

显示翻译

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

我把一种全新的编程方式称为“氛围编程”，在这种模式下，你完全沉浸在编程的氛围里，拥抱指数级的效率提升，甚至忘了代码本身的存在。之所以能做到这一点，是因为大语言模型（比如搭载 Sonnet 的 Cursor Composer）已经变得太过强大了。而且我只用 SuperWhisper 和 Composer 交流，几乎都不用碰键盘。我会提一些特别简单的需求，比如“把侧边栏的内边距减半”，因为我懒得自己去找这个设置。我总是直接“全部接受”生成的代码，再也不看代码差异了。遇到报错信息时，我就直接复制粘贴进去，什么都不说，通常这样就能解决问题。写出来的代码已经超出了我以往的理解范围，得花时间仔细通读才能看懂。有时候大语言模型修不好某个漏洞，我就换个方法绕过去，或者让它做一些随机的修改，直到漏洞消失。这种方式做一次性的周末小项目完全没问题，而且还挺有意思的。我现在正在做一个项目或网页应用，但这已经算不上真正的编程了——我只是看看内容、说点指令、运行程序、复制粘贴内容，结果大多都能正常运行。

上午7:17 · 2025年2月3日 ·

688.1万 查看

多轮对话

类比AI对话框

我们平时使用AI对话框的时候，也经常发现大模型一次性返回的内容不符合我们的预期，于是我们会采用追问的方式，或者补充更多信息的方式进行多轮对话。

快手翻译成英文

The translation of "快手" into English is **Kuaishou**.

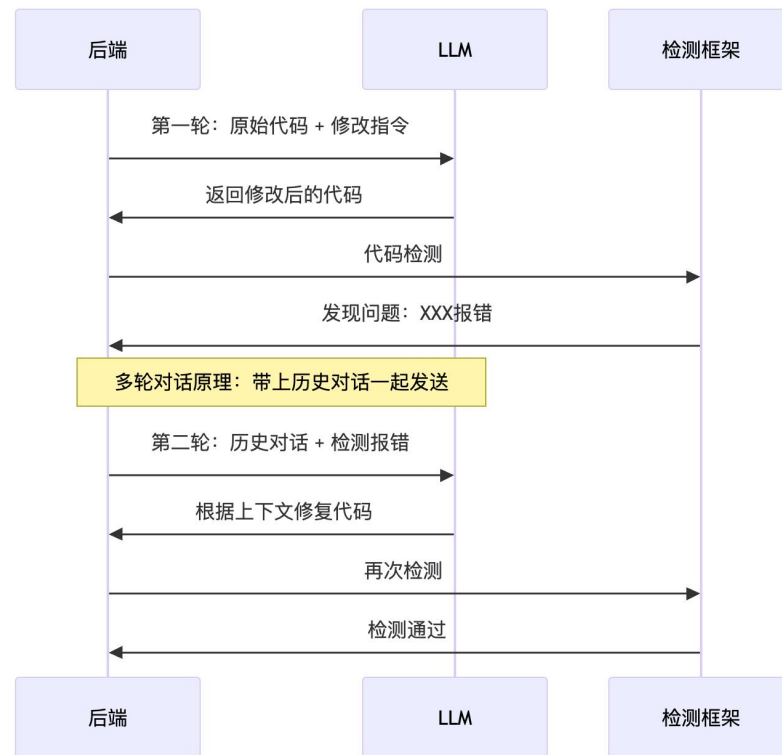
If you are looking for the name of the app as it is marketed internationally, it is called **Kwai**.

只给我输出结果就行，不用解释

Kuaishou



基于Session的多轮对话实现



● 校验模块强卡控



有了多轮对话，如何检测
AI修改是否有问题？



逻辑检查插件

- 是否下线了多余开关
- 布尔逻辑表达式是否被改反
- 开关关联的业务逻辑是否完整
- 是否误删非开关相关代码



大模型修改后强校验框架

通过扩展检测插件，全方位检测，拦截各类Bad Case



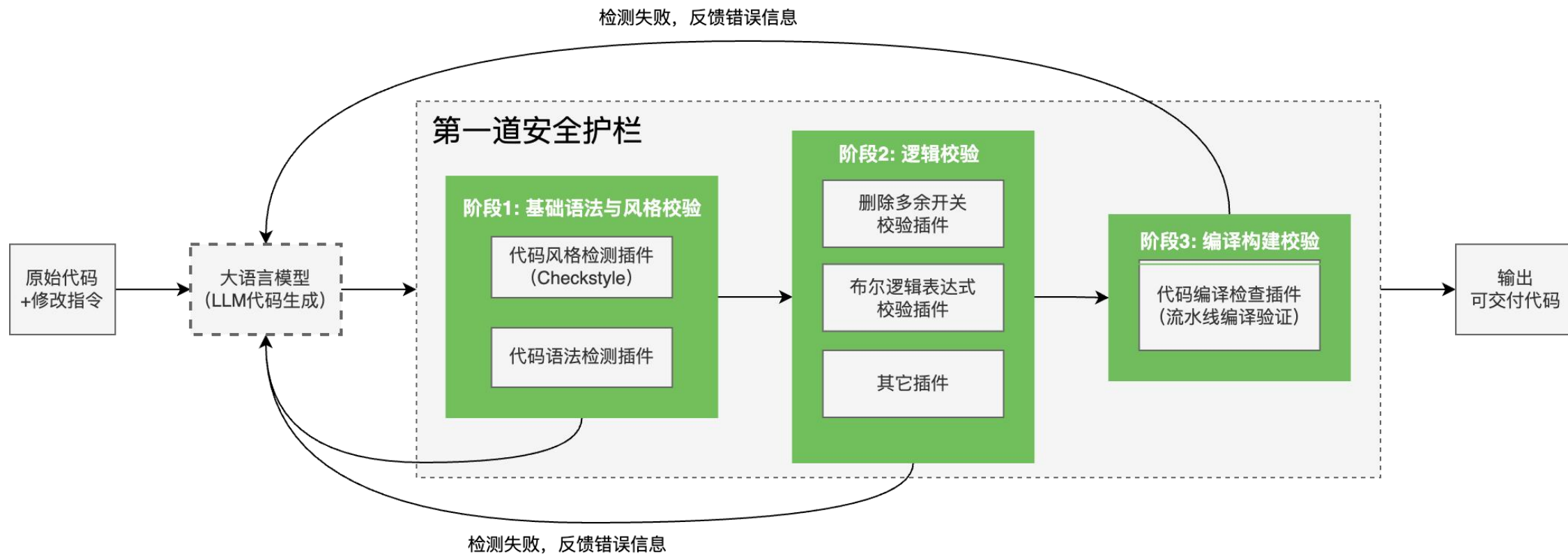
编译检查插件

- 代码格式是否符合CheckStyle规范
- 是否存在语法错误
- 流水线编译是否能通过

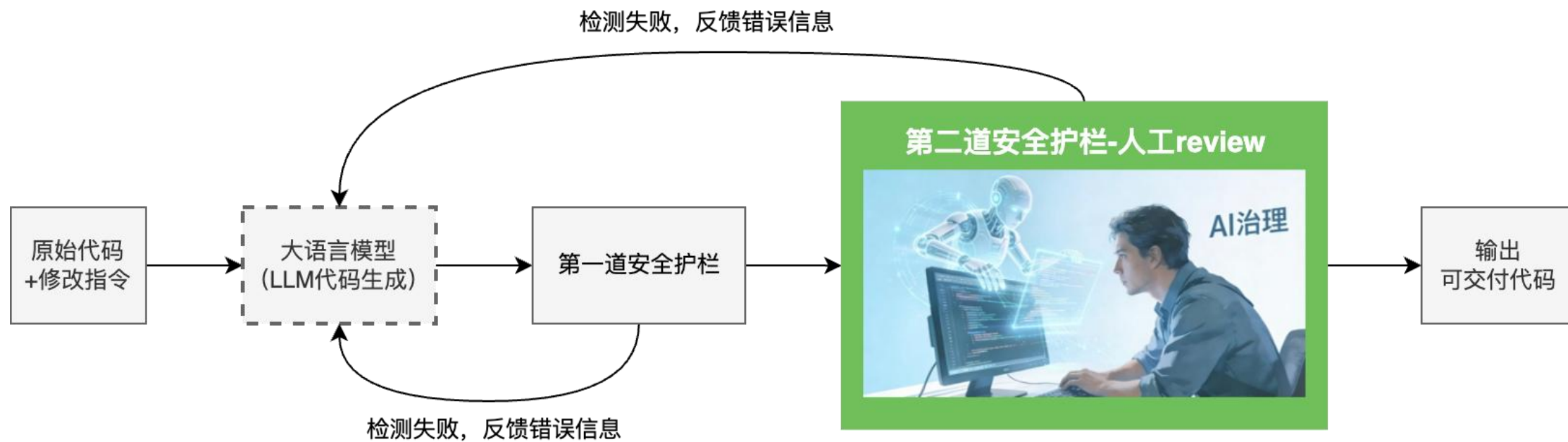
检测通过 → 进入下一轮 or 提交MR；检测失败 → 反馈错误给AI迭代



第一道安全护栏 - 编译/逻辑错误拦截



第二道安全护栏 - 人工拦截兜底



大模型检查



评审模型独立推理

3个互不依赖的LLM分开判断，不共享上下文偏见

投票胜出

2个或2个以上通过，才通过

⚠ 只要Review结果的确定性达不到100%，所有的修改结果还是需要100%的人工参与度

PAL: Program-aided Language Models

Luyu Gao^{*1} Aman Madaan^{*1} Shuyan Zhou^{*1} Uri Alon¹ Pengfei Liu^{1,2}
Yiming Yang¹ Jamie Callan¹ Graham Neubig^{1,2}
{luyug, amadaan, shuyanzh, ualon, pliu3, yiming, callan, gneubig}@cs.cmu.edu

Abstract

Large language models (LLMs) have demonstrated an impressive ability to perform arithmetic and symbolic reasoning tasks, when provided with a few examples at test time (“few-shot prompting”). Much of this success can be attributed to prompting methods such as “chain-of-thought”, which employ LLMs for both *understanding* the problem description by decomposing it into steps, as well as *solving* each step of the problem. While LLMs seem to be adept at this sort of step-by-step decomposition, LLMs often make logical and arithmetic mistakes in the solution part, even when the problem is decomposed correctly. In this paper, we present Program-Aided Language models (PAL): a novel approach that uses the LLM to read natural language problems and generate *programs* as the intermediate reasoning steps, but offloads the *solution* step to a runtime such as a Python interpreter. With PAL, decomposing the natural language problem into runnable steps remains the only learning task for the LLM, while solving is delegated to the interpreter. We demonstrate this synergy between a neural LLM and a symbolic interpreter across 13 mathematical, symbolic, and algorithmic reasoning tasks from BIG-Bench Hard and others. In all these *natural language* reasoning tasks, generating code using an LLM and reasoning using a Python interpreter leads to more accurate results than much larger models. For example, PAL using CODEX achieves state-of-the-art few-shot accuracy on GSM8K, surpassing PaLM-540B which uses chain-of-thought by absolute 15% top-1.¹

¹Equal contribution ²Language Technologies Institute, Carnegie Mellon University, USA ³Inspired Cognition, USA. Correspondence to: Luyu Gao, Aman Madaan, Shuyan Zhou <{luyug, amadaan, shuyanzh}@cs.cmu.edu>.

Proceedings of the 40th International Conference on Machine Learning, Honolulu, Hawaii, USA. PMLR 2022, 2023. Copyright 2023 by the author(s).

¹Code and data at <http://reasonwithpal.com>.

1. Introduction

Until as recently as three years ago, reasoning was considered to be one of the most significant challenges that large language models (LLMs) had not yet overcome (Marcus, 2018; 2020; Garcez & Lamb, 2020). Recently, LLMs have shown impressive success on a wide range of reasoning tasks, including commonsense (Wei et al., 2021; Sanh et al., 2021; Madaan et al., 2022), mathematical (Lewkowycz et al., 2022; Wu et al., 2022; Mishra et al., 2022), and symbolic reasoning (Yao et al., 2022; Ahn et al., 2022), using few-shot prompting (Brown et al., 2020).

This process has been accelerated by methods that require LLMs to generate their explicit reasoning steps, such as “chain-of-thought” (Wei et al., 2022), “scratchpads” (Nye et al., 2021a), and “least-to-most” (Zhou et al., 2022) prompting. In particular, the widely used chain-of-thought (CoT) method presents the model with the explicit intermediate steps that are required to reach the final answer. Then, the model is expected to apply a similar decomposition to the actual test example, and consecutively reach an accurate final answer (Ling et al., 2017; Amini et al., 2019). Nevertheless, while LLMs can decompose natural language problems into steps and perform *simple* arithmetic operations, their performance falls dramatically when dealing with complex arithmetic (Hendrycks et al., 2021; Madaan & Yazdanbakhsh, 2022) or large numbers (Geva et al., 2020; Nogueira et al., 2021; Qian et al., 2022). In fact, even when fine-tuning a PaLM-based model on 164B tokens of explicit mathematical content, its two most common failures are reportedly “incorrect reasoning” and “incorrect calculation” (Lewkowycz et al., 2022).

In this paper, we propose Program-Aided Language model (PAL): a novel approach that uses an LLM to read natural language problems and generate *programs* as reasoning steps, but offloads the *solution* step to a Python interpreter, as illustrated in Figure 1. This offloading leverages an LLM that can decompose a natural language problem into programmatic steps, which is fortunately available using contemporary state-of-the-art LLMs that are pre-trained on both natural language and programming languages (Brown et al., 2020; Chen et al., 2021a; Chowdhery et al., 2022). While

PAL: 程序辅助语言模型

陆宇高^{*1} 阿南·马丹¹ 周书彦^{*1} Uri Alon¹ 刘鹏飞^{1,2} 杨一鸣¹ 杰米·卡伦¹ 格雷厄姆·纽比格^{1,2}

{luyug, amadaan, shuyanzh, ualon, pliu3, 一鸣, callan, gneubig}@cs.cmu.edu

摘要

大型语言模型 (LLMs) 在测试时提供少量示例 (即 “少样本提示”) 的情况下, 已展现出出色的算术和符号推理任务能力。这一成功很大程度上归功于 “思维链” 等提示方法, 这类方法利用 LLM 通过将问题分解为步骤来理解问题描述, 并解决问题的每一个步骤。尽管 LLMs 似乎擅长此类步步分解, 但即便问题分解正确, LLM 在求解环节仍常出现逻辑和算术错误。本文提出了程序辅助语言模型 (PAL): 这是一种新颖的方法, 利用 LLM 阅读自然语言问题并生成程序作为中间推理步骤, 同时将求解步骤交由 Python 解释器等运行时环境处理。在 PAL 框架中, 将自然语言问题分解为可执行步骤仍是 LLM 的唯一学习任务, 而求解则委托给解释器完成。我们针对 BIG-Bench Hard 及其他数据集的 13 项数学、符号和算法推理任务, 验证了神经 LLM 与符号解释器之间的这种协同效应。在所有这些自然语言推理任务中, 利用 LLM 生成代码并通过 Python 解释器进行推理, 所取得的结果准确率远高于规模大得多的模型。例如, 采用 CODEX 的 PAL 在 GSM8K 数据集上实现了最先进的少样本准确率, 超越了 PaLM-540B 模型

在思维链 (chain-of-thought) 的基础上, top-1 准确率绝对提升了 15%。¹

^{*} 同等贡献 ¹ 美国卡内基梅隆大学语言技术研究所 ² 美国灵感认知公司。通讯作者: 高路宇、阿南·马丹、周诗妍, 邮箱: <{luyug, amadaan, shuyanzh}@cs.cmu.edu>。

第 40 届国际机器学习大会会议论文集, 美国夏威夷檀香山。PMLR 2022, 2023 年。版权所有 ©2023 年作者本人。
¹ 代码与数据见 <http://reasonwithpal.com>。

1. 引言

就在三年前, 推理能力还被认为是大型语言模型 (LLMs) 尚未攻克的最重大挑战之一 (Marcus, 2018; 2020; Garcez & Lamb, 2020)。近年来, 大型语言模型借助少样本提示 (Brown et al., 2020), 在各类推理任务中取得了令人瞩目的成果, 这些任务包括常识推理 (Wei et al., 2021; Sanh et al., 2021; Madaan et al., 2022)、数学推理 (Lewkowycz et al., 2022; Wu et al., 2022; Mishra et al., 2022) 以及符号推理 (Yao et al., 2022; Ahn et al., 2022)。

通过要求大语言模型 (LLMs) 生成明确推理步骤的方法, “思维链” (Wei 等人, 2022)、“草稿本” (Nye 等人, 2021a) 以及 “从少到多” (Zhou 等人, 2022) 提示法, 进一步加速了这一过程。具体而言, 应用广泛的思维链 (CoT) 方法会向模型展示得出最终答案所需的明确中间步骤。随后, 模型需对实际测试示例采用类似的分解方式, 进而连续得出准确的最终答案 (Ling 等人, 2017; Amini 等人, 2019)。然而, 尽管大语言模型能够将自然语言问题分解为多个步骤并完成简单算术运算, 但在处理复杂算术 (Hendrycks 等人, 2021; Madaan & Yazdanbakhsh, 2022) 或大数 (Geva 等人, 2020; Nogueira 等人, 2021; Qian 等人, 2022) 时, 其性能会急剧下降。事实上, 据报道, 即便在基于 PaLM 的模型上对 1640 亿个显式数学内容标记进行微调后, 其两种最常见的失败类型仍是 “推理错误” 和 “计算错误” (Lewkowycz 等人, 2022)。

在本文中, 我们提出了程序辅助语言模型 (PAL): 这是一种新颖的方法, 利用大语言模型 (LLM) 解读自然语言问题并将推理步骤生成成为程序, 同时将求解步骤交由 Python 解释器执行, 如图 1 所示。这种任务分流的方式依托于大语言模型的能力——它能够将自然语言问题拆解为程序化步骤, 而这一点在当前最先进的、经自然语言和编程语言联合预训练的大语言模型上均已实现 (Brown 等人, 2020; Chen 等人, 2021a; Chowdhery 等人, 2022)。尽管

AST引擎：用确定性的程序检测代替人工兜底复核

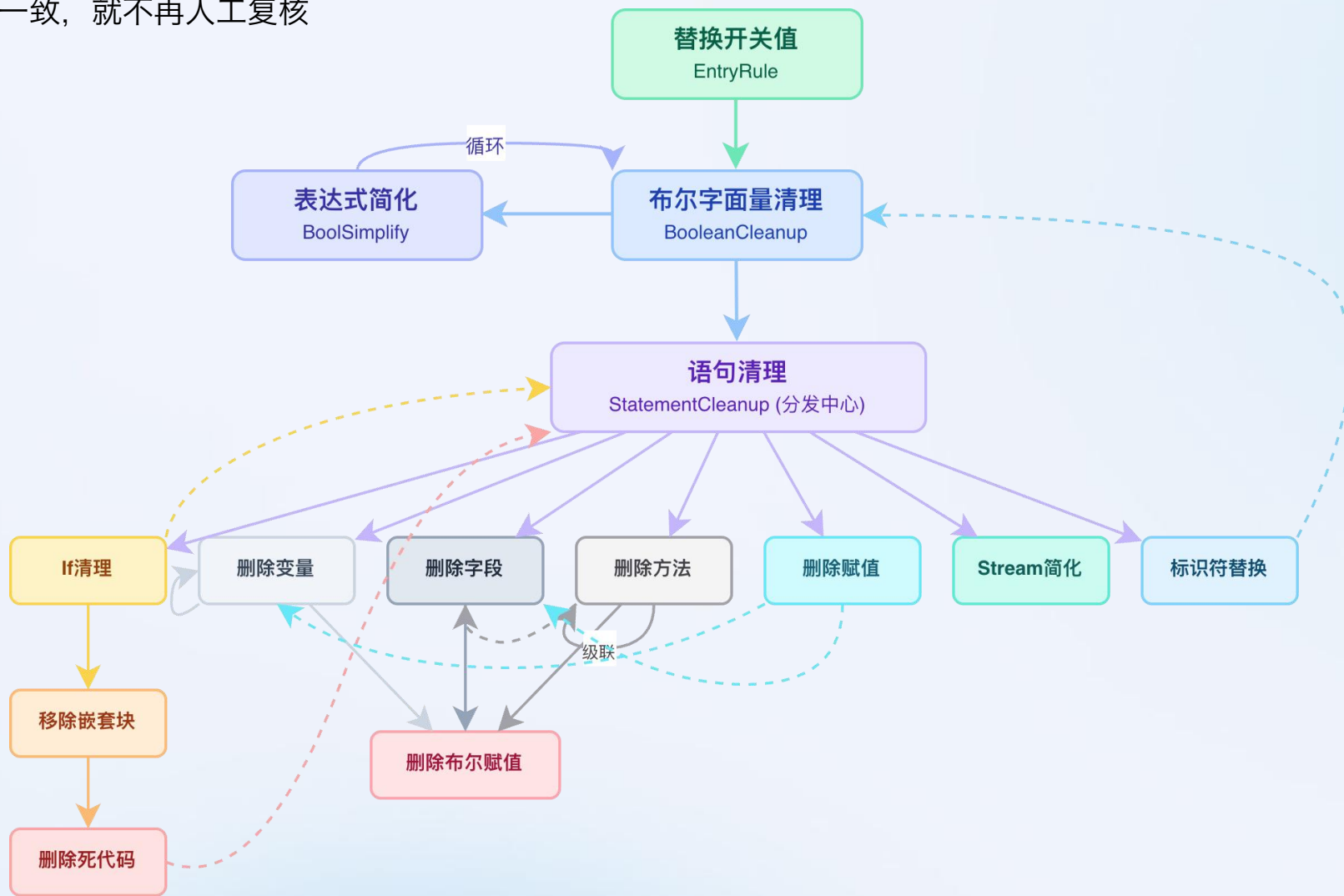
打造AST引擎，如果和AI引擎修改的结果一致，就不再人工复核

规则

规则原子化，一个规则只做一件事情

有向图

有向图驱动流程，级联触发规则



LLM生成+AST校验 双引擎架构

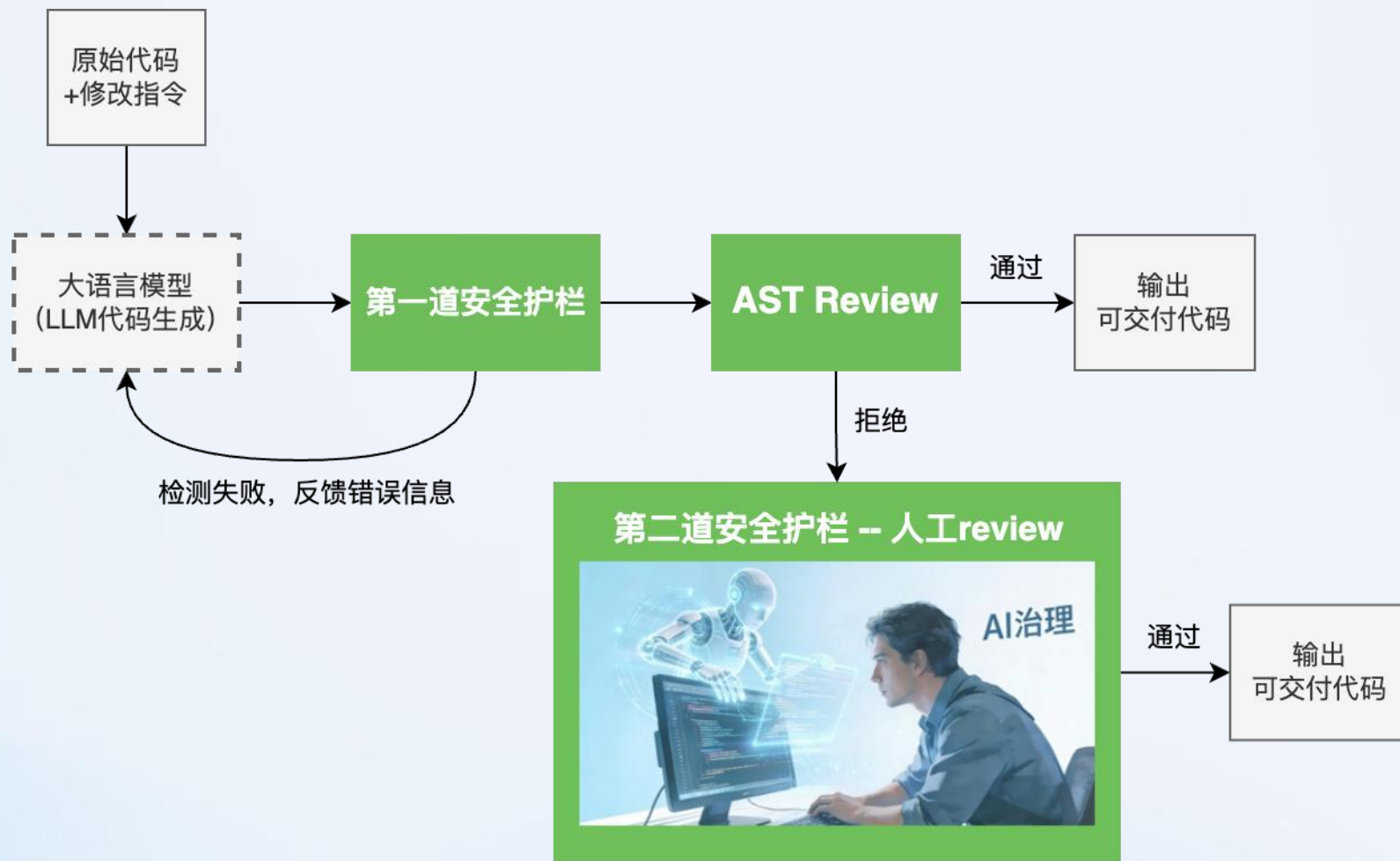


LLM大模型：
处理模糊性，生成创新方案

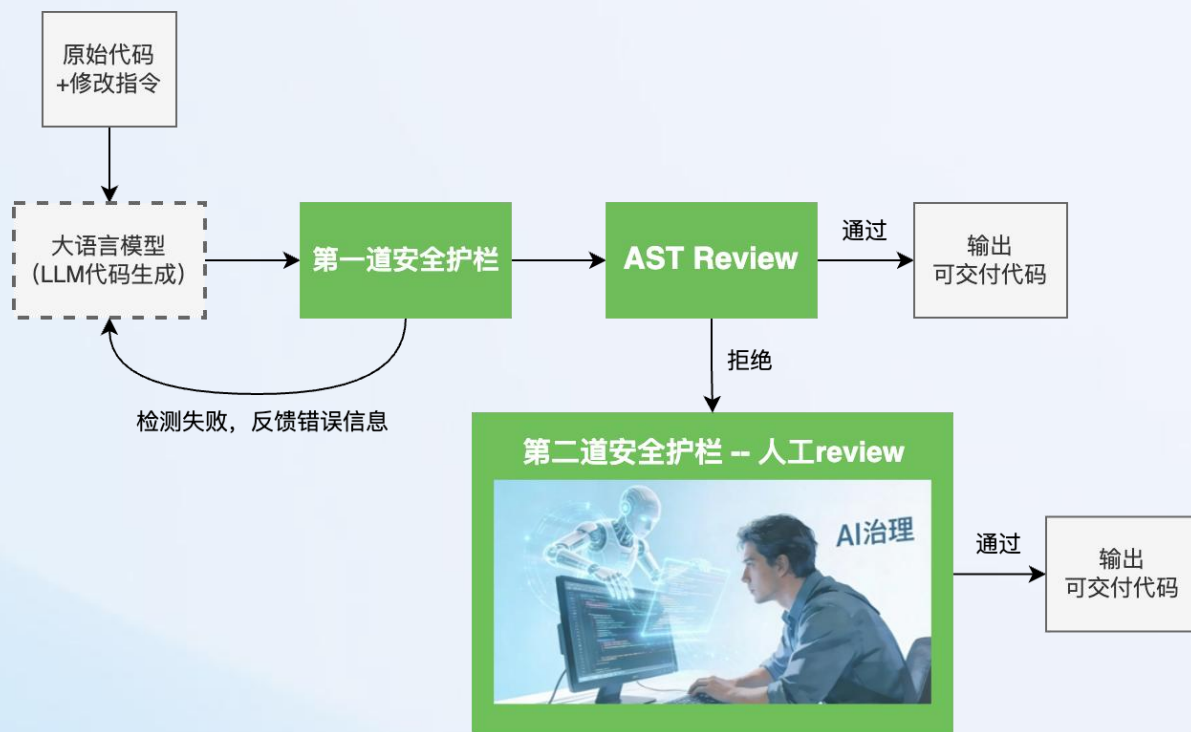


AST校验：
程序规则确保零误判准入

治理Agent的整体流程



AST一定是正确的吗?



流水线测试兜底

集成测试

流量录制回放diff

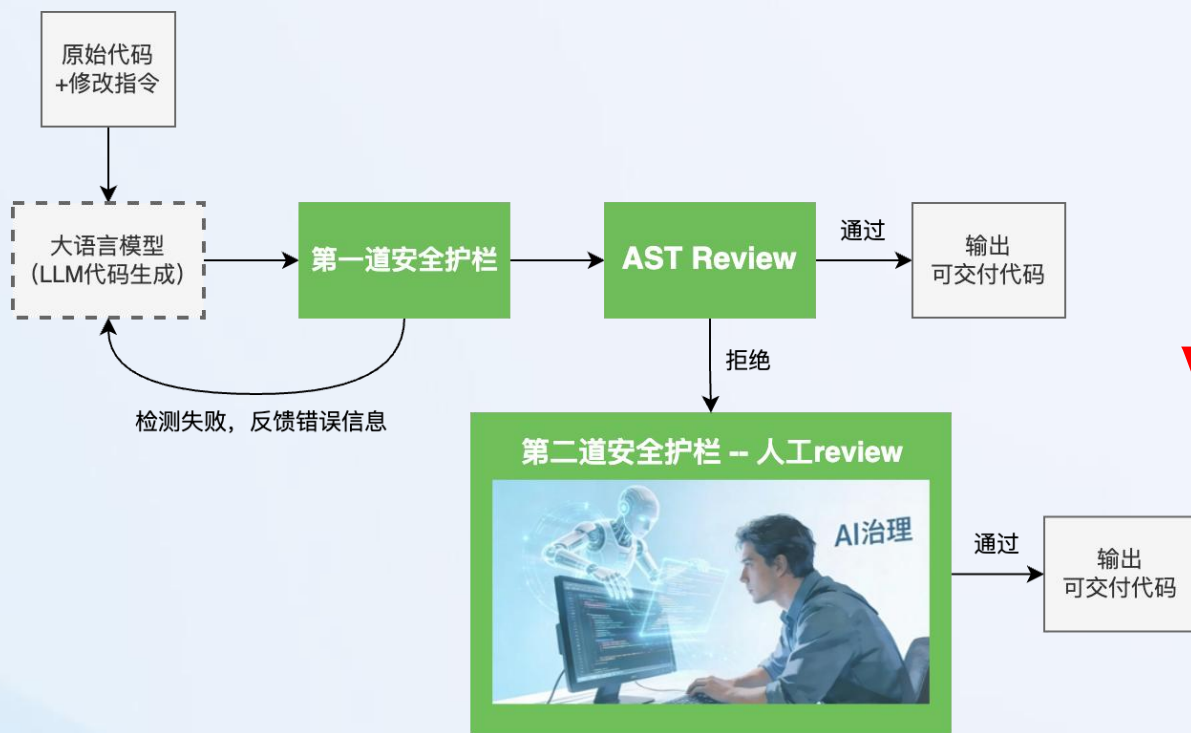
将治理压力由业务转向平台

平台不是只提供提效工具，也要承担责任
之前是业务有bug，改错了，业务自己负责
现在是AST有bug，平台负责

AST引擎和AI引擎同时改错的概率几乎为0

两者互补，且互相反哺

有AST引擎了，还需要AI引擎吗？



VS



AST不支持的代码一定会改错，到时候无法发现，那就还是需要人工100%的review

03

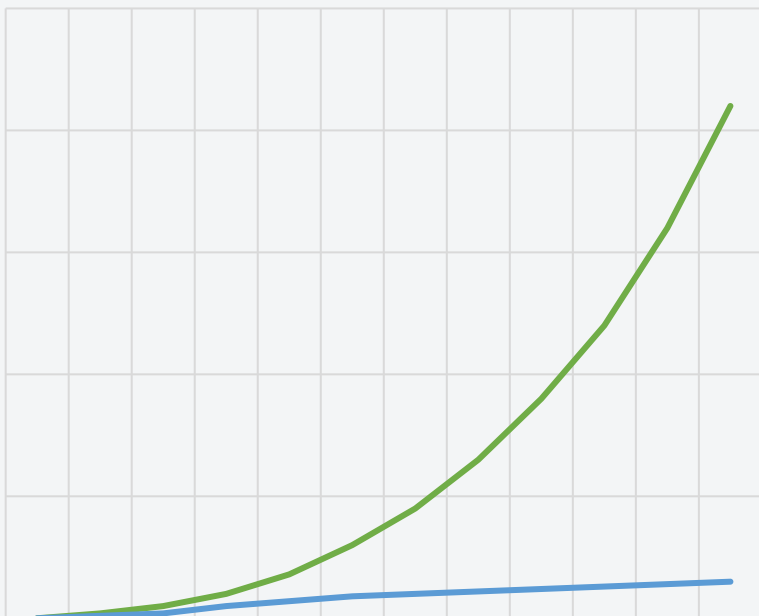
自我进化：人工到系统自升级

让治理系统具备自我迭代、自我修复、自我优化的能力

治理系统的进化刚需：规模一上来，人工维护就失效

规模增长 vs 人工维护能力

— 治理需求 — 人工维护能力



人工维护带来的三大困境

1

效率低

人工Review排队，问题积压

2

人力成本浪费

AI大部分改的都是正确的，人力投入在无效review

3

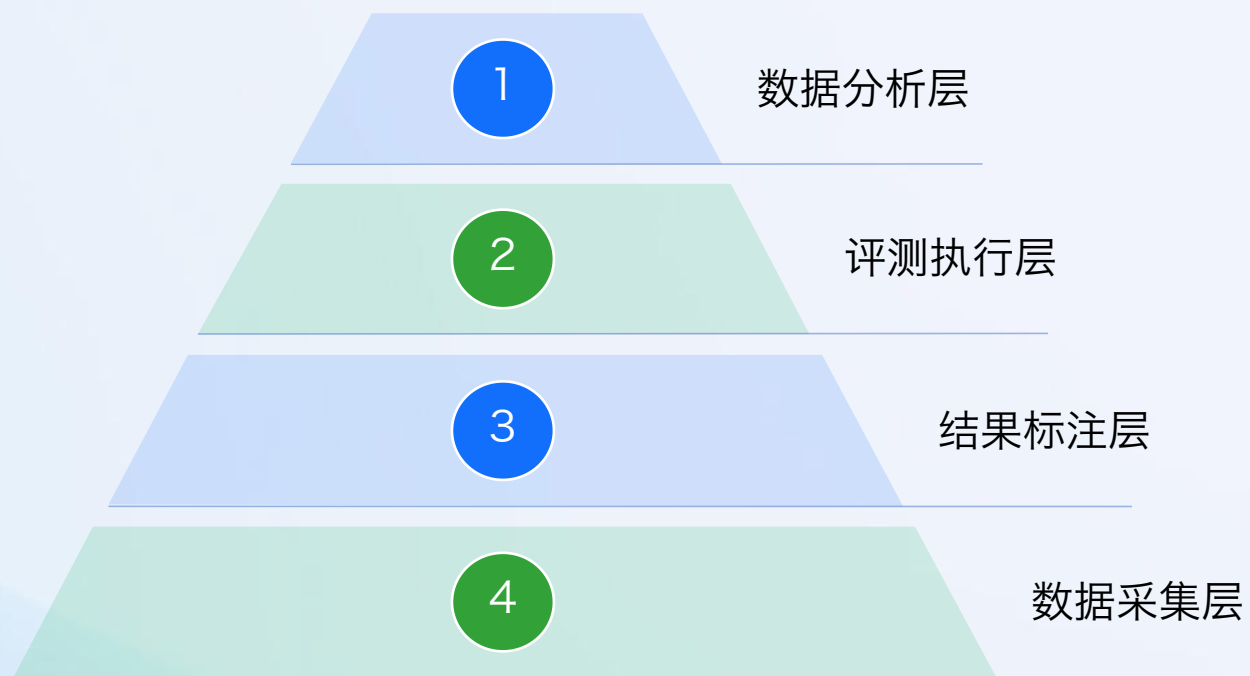
滞后强

AST引擎和校验插件更新速度慢

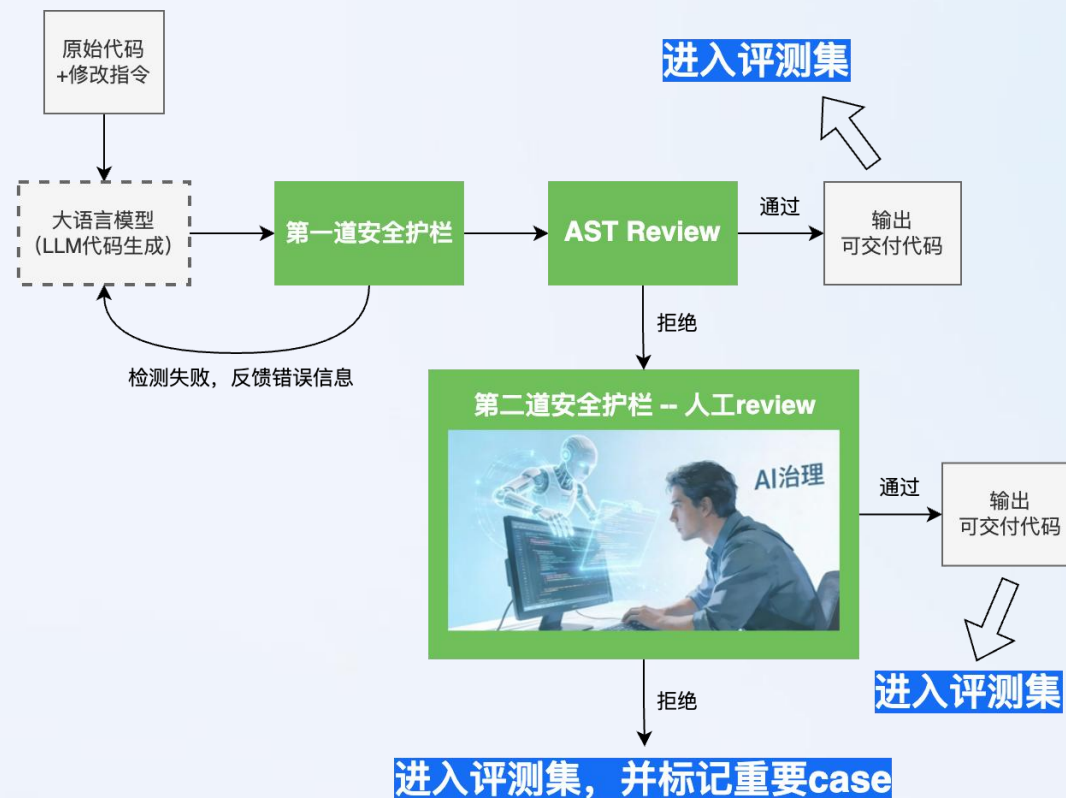
评测体系：让优化有“据”可依



● 评测体系：让优化有“据”可依



评测系统的技术架构



数据采集 + 结果标注

● 让每一次人工复核，都变成治理系统进化的燃料

人工标注 -> 系统自优化

人工对治理结果进行标注

优化完成 -> 自动评测

系统自动进化

持续迭代
正向飞轮

评测通过 -> 减少人工标注

通过评测会使正确率提升
正确率提升会减少人工标注的量

让每一次人工复核，都变成治理系统进化的燃料

⚠ 漏检：人工复核后AI是**错的**

本质 检测插件能力不足，有盲区未覆盖（检测缺口），规则强度不足

➔ **进化方向**

把漏过的风险类型沉淀为可复用的检测能力



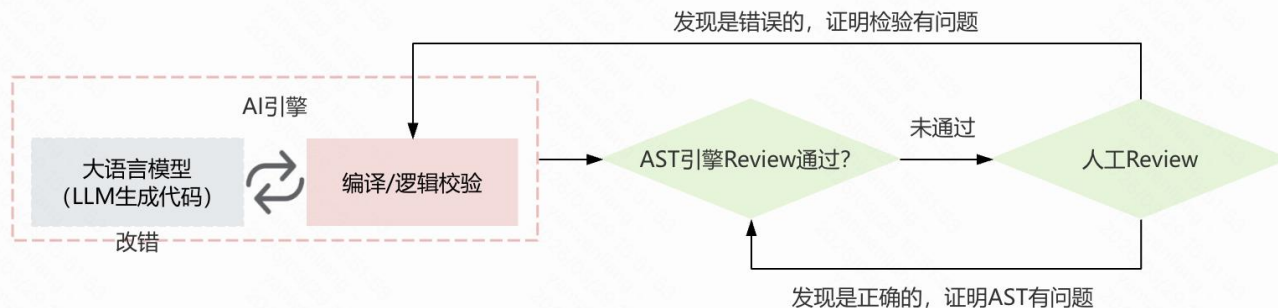
如何进化

⊘ 误判：人工复核后AI是**对的**

本质 AST规则僵化，误判率高
误伤正确变更，识别不足

⬅ **进化方向**

把“正确但被推给人工”的案例沉淀为规则



双Agent专项升级





The screenshot displays the 'AST能力升级' (AST Capability Upgrade) interface. At the top, there's a navigation bar with tabs for '理解需求文档' (Understand Requirement Document), '编写技术方案' (Write Technical Solution), and '编写代码' (Write Code). Below this, a status bar indicates 'AST能力升级' and 'AST代码已发生变化. 请关注后续流程中节点的动作/产物是否需要重新执行或修改' (AST code has changed. Please pay attention to the actions/products of the nodes in the subsequent process flow that may need to be re-executed or modified).

The main content area is titled '标准PRD' (Standard PRD) and shows a code snippet for a PRD. The code is as follows:

```
boolean holdoutUser = HoldoutUtils.isSocialMessageHoldoutUser(userId);
return holdoutUser ? false : true;
```

Below the code, there's a section for '修改后:' (After Modification) with the following code:

```
return !HoldoutUtils.isSocialMessageHoldoutUser(userId);
```

On the right side, there's a chat window titled '需求理解chat' (Requirement Understanding Chat). The chat content includes:

- 让我查看DESIGN_TODO.md, 看看是否有相关的设计文档:
- 现在让我搜索与代码简化、表达式优化相关的内容, 特别关注三元表达式和返回语句的处理:
- 现在让我查看pom.xml了解项目依赖, 以及查找是否有关于表达式简化的现有代码:
- 根据原始PRD和代码仓库分析, 我现在可以生成技术导向的需求总结了。

The chat window also shows a '更新文档' (Update Document) button and a status bar indicating '已生成' (Generated).

On the left side, there's a sidebar with two sections:

- 涉及范围** (Scope):
 - 需要扩展现有的AST代码生成或后处理能力
 - 可能涉及ZeroHomersEngine (外部依赖) 或在ZeroHomersimpl中添加后处理逻辑
 - 需要在CodeAnalysisUtils中增加检测和优化此类模式的方法
- 技术要点** (Key Points):
 - 识别【变量 ? false : true】和【变量 ? true : false】模式
 - 检测并消除不必要的临时变量
 - 简化布尔表达式:
 - 【条件 ? false : true】→ 【条件】
 - 【条件 ? true : false】→ 【条件】
 - 保证简化后的代码语义等价
 - 确保优化不影响代码的语法正确性

分支

已完成 AST能力升级

理解需求文档

编写技术方案

编写代码

研发产出

分支 的代码已发生变化，请关注后续流程中节点的动作/产物是否需要重新执行或修改

技术方案

2.2 核心优化策略

本次优化聚焦于识别和简化以下布尔表达式模式：

原始模式	优化结果	说明
<code>condition ? false : true</code>	<code>!condition</code>	反向逻辑
<code>condition ? true : false</code>	<code>condition</code>	保持逻辑
变量 = 表达式; return 变量 ? x : y	return 表达式 ? x : y	消除临时变量

2.3 优化流程时序图

图片 代码 全屏

```
sequenceDiagram
    participant ZHI as ZeroHomersImpl
    participant ZHE as ZeroHomersEngine
    participant BEO as BooleanExpressionOptimizer
    participant JPU as JavaParserUtils
    participant CAU as CodeAnalysisUtils

    ZHI->>ZHE: removeFlagByAST()
    ZHE-->>ZHI: astGeneratedCode
    Note over ZHI: 新增优化环节
    ZHI->>BEO: optimize(astGeneratedCode)
    BEO->>JPU: parse(code)
```

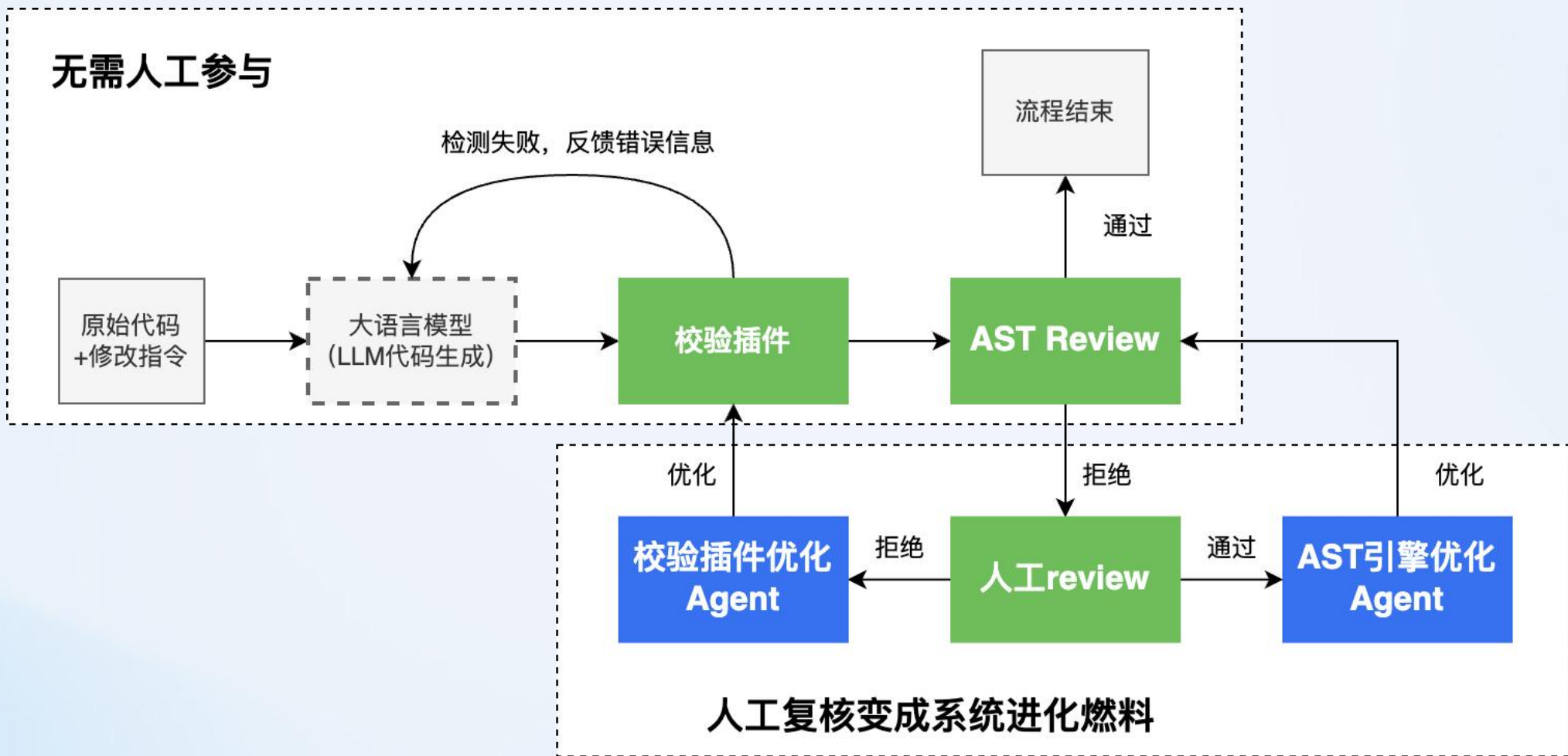
技术方案的编写是开发流程中至关重要的一环，它不仅是开发团队沟通的桥梁，更是项目成功的关键。在编写技术方案时，我们应遵循以下原则：

- 清晰明了：使用简洁的语言和图表，清晰地表达技术思路。
- 逻辑严谨：确保方案的逻辑严密，经得起推敲。
- 可落地性：方案应具有可操作性，能够指导开发工作。
- 沟通协作：编写过程中应与团队成员保持密切沟通，确保方案符合实际需求。

通过遵循这些原则，我们可以编写出高质量的技术方案，为项目的顺利推进提供有力支持。



整体流程



● 整体架构

开关治理Agent整体架构

工具层

代码检索

源代码拉取

提交MR

流水线调用

检测层

编译检测

逻辑检测

AST检测

大模型检测

评测

结果标注



看板



回测



报告

AI基建

LLM

会话存储

多轮对话

模型切换

提示词

自进化
Agent



AI Native: 全生命周期



之前工作的局限

之前做的所有工作，其实都集中在开关生命周期的**最后一个环节**——“删除”，这是很片面的

⚠️ 真正的AI Native开关治理，应该**覆盖开关的全生命周期**



智能创建

在需求研发阶段，就让AI参与进来

- ✓ AI根据需求场景，判断是否需要使用开关
- ✓ 判断开关的作用（灰度放量、功能回滚等）
- ✓ 给开关打上分类标签
- ✓ 预设开关的下线时间和条件

💡 把脑子里的经验，沉淀下来作为AI的上下文

- ✓ 创建之初，就具备“可自动治理”的属性



智能变更

AI参与变更计划，变更巡检，变更阻断

- ✓ 智能灰度放量：根据风险等级，自动分级
- ✓ 智能自动巡检，变更过程巡检系统/业务指标
- ✓ 异常自动回滚
- ✓ 动态调权与智能熔断

📋 AI参与变更，避免人工忘记放量和巡检

- ✓ 人工变更伙伴



智能删除

基于创建时的上下文，自动删除

- ✓ 根据开关的预设条件
- ✓ 根据业务迭代情况
- ✓ 根据用户使用数据
- ✓ 自动判断开关是否已经无用

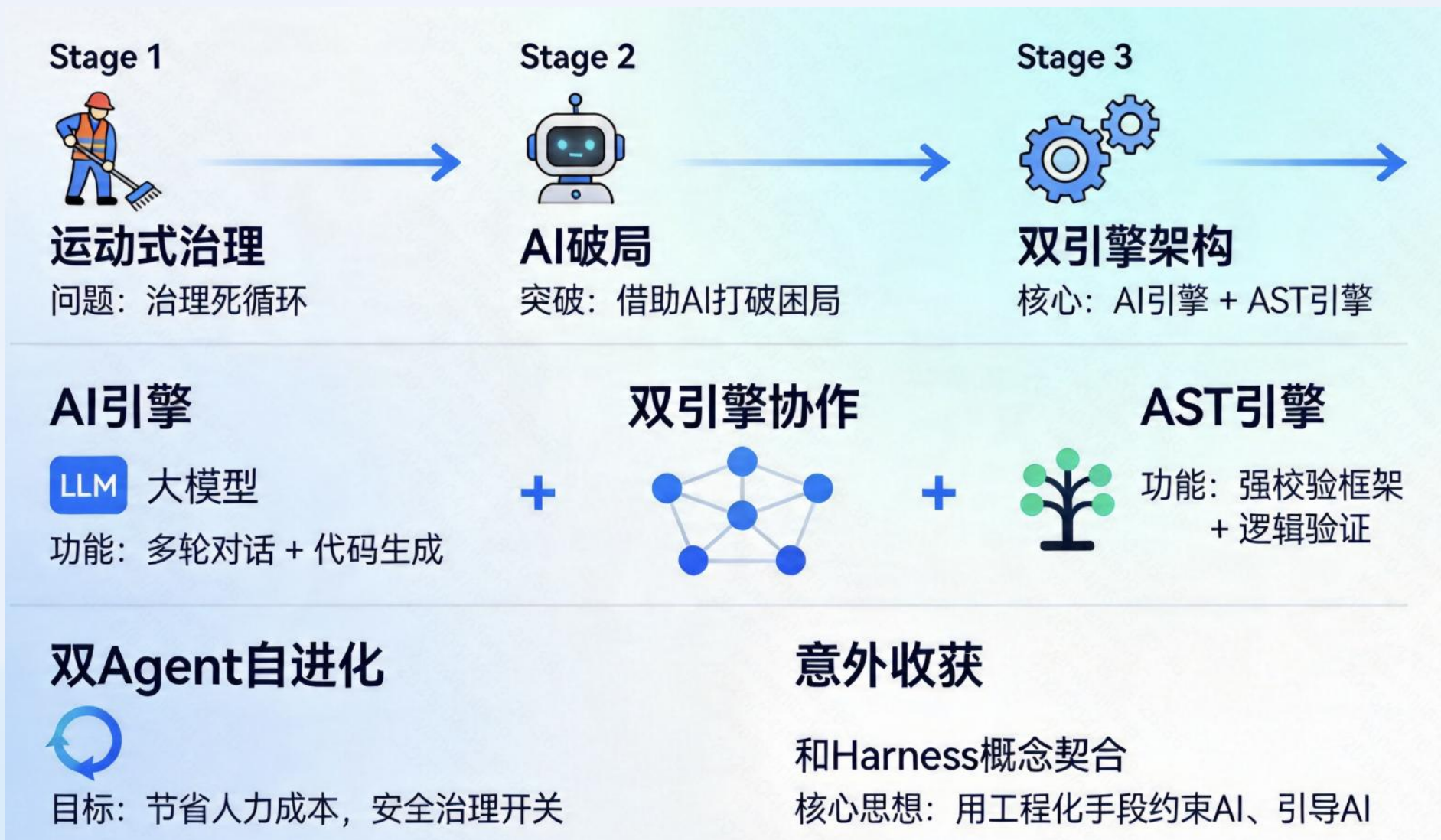
🔧 无需人工介入，自动完成下线 and 代码清理

- ✓ 实现开关的“自我消亡”

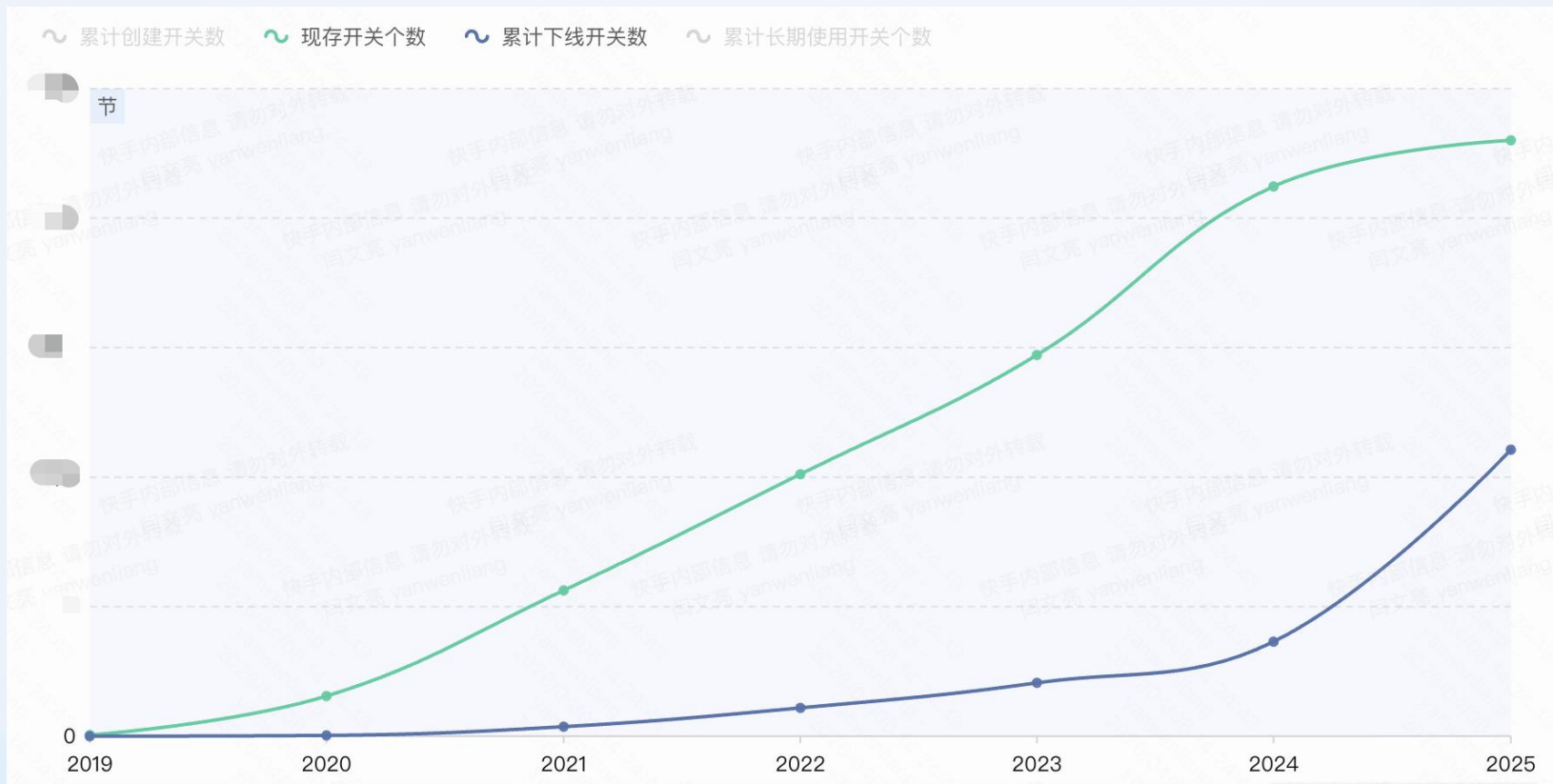
04

总结与展望

● 实践总结: Harness Engineering (驾驭工程)

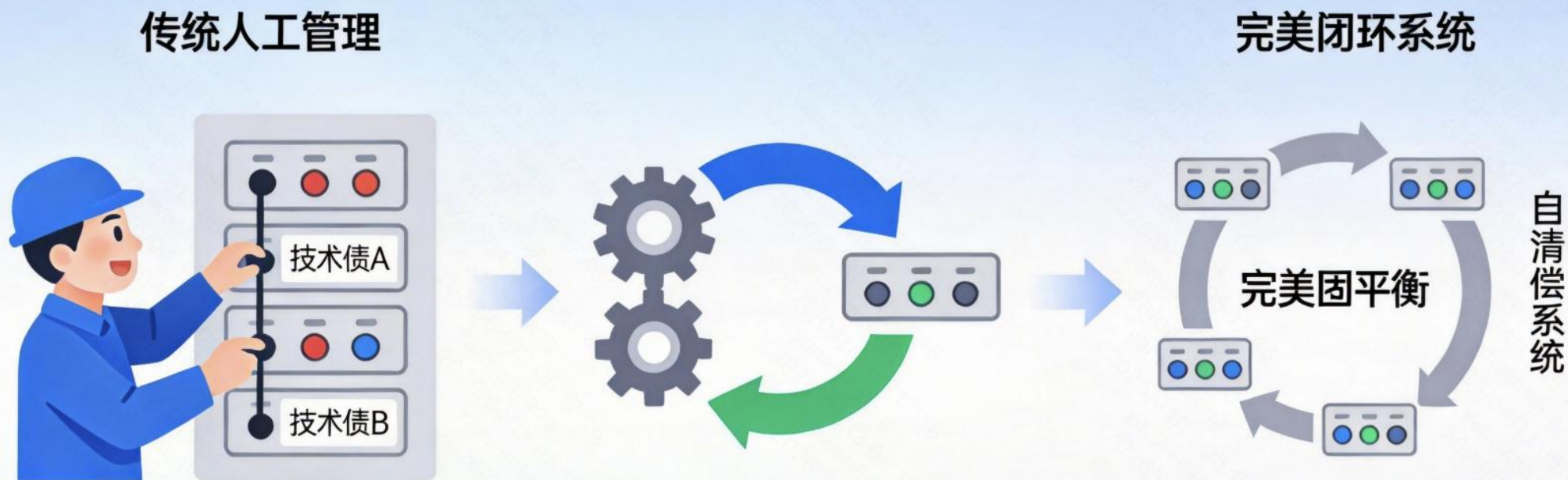


AI累计下线1500+开关，删除代码60000+行



AI治理当前准确率在98%以上，AST引擎和AI引擎的拟合率在80%以上

AI时代的工程治理新范式



“最好的治理，是治理本身被遗忘；最好的系统，是系统自己照顾自己”

AI生成

QCon
全球软件开发大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



ywl

中国大陆



扫一扫上面的二维码图案，加我为朋友。

AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

